React

=> Javascript library for building user interfaces.

=> Open source, component-based front end library responsible only for the view layer of the application.

=> Component based Architecture.

=> Reusable code.

=> React is declarative

⇓

C: Basically we just have to tell react what we want & react with its react dom library will build actual UI)

Features of React

1. JSX (Javascript XML)
2. Components
3. Virtual DOM.
4. One way data binding
5. High Performance (react update only those comp. instead of updating all comp).

Scanned with OKEN Scanner

Why react

=> Improved performance (virtual DOM)

=> reusable components

=> Unidirectional data flow. (helps easy to debug & find error).

=> dedicated tools.

=> easily readable

=> light weight.

Class component Functional Component

State

=> sources of data and must be kept as simple as possible.

=> state will determine component rendering and behaviour.

=> mutable & creative dynamic & interactive components.

Synthetic events

=> Cross browser wrapper.

-> React normalizes event so that they have consistent properties across browsers ~~browser contact~~

Pass child to Parent

=> Create a call back fn in parent comp. This callback fn ll get data from the child component.

=> pass a callback fn to the child as a prop.

=> child component calls the parent callback fn as prop.

Shallow Comparison

-> Primary type.

(if both the value of string is same return true).

2) Complex type.

Both the reference of objects/ array are same, then it ll return true.

Ref => To access the DOM nodes or react elements created in render method.
=> To access the child method in parent.

Ref => ex:- ~~increment~~ the useRef ll return empty current : {} obj.

To access DOM nodes or react elements created in render method.

Pure components and memo is same.

React.memo(component) like HOC.

useMemo => useMemo only runs when only of the dependencies update => improve perform

returns a memoised value. Its memoise so no need to re-calculate

```
useMemo(() => {
    return slowFunction(number)
}, [number])
```

State or props doesn't change we don't need to re-render or recompute.

One thing to note

=> why we are not memoise ~~all the things~~ everything.

=> Unnecessarily we are saving the previous value in some memory variable.

=> So we are forcing memory to get larger everytime we use memo bcaz we have to store additional variable in memory to store the previous value.

=> If we use in unnecessary place then memory will be wasted & performance also ll be slow

=> Use memo only when fn is incredibly slow.

useRef

=> To get the previous value of the state.

useRef

const variable = useRef(1). | const variable = React.useRef().

variable.current | variable.current

(or)

" .current.focus

Portals

=> provide a way to render children into a DOM node ~~that exists~~ outside ~~the DOM hierarchy~~ of parent DOM hierarchy component.

=> Sometimes it is useful to insert child into different location in the DOM.

return React.createPortal((

JSX
DOM node] -> two parameter

to mount element

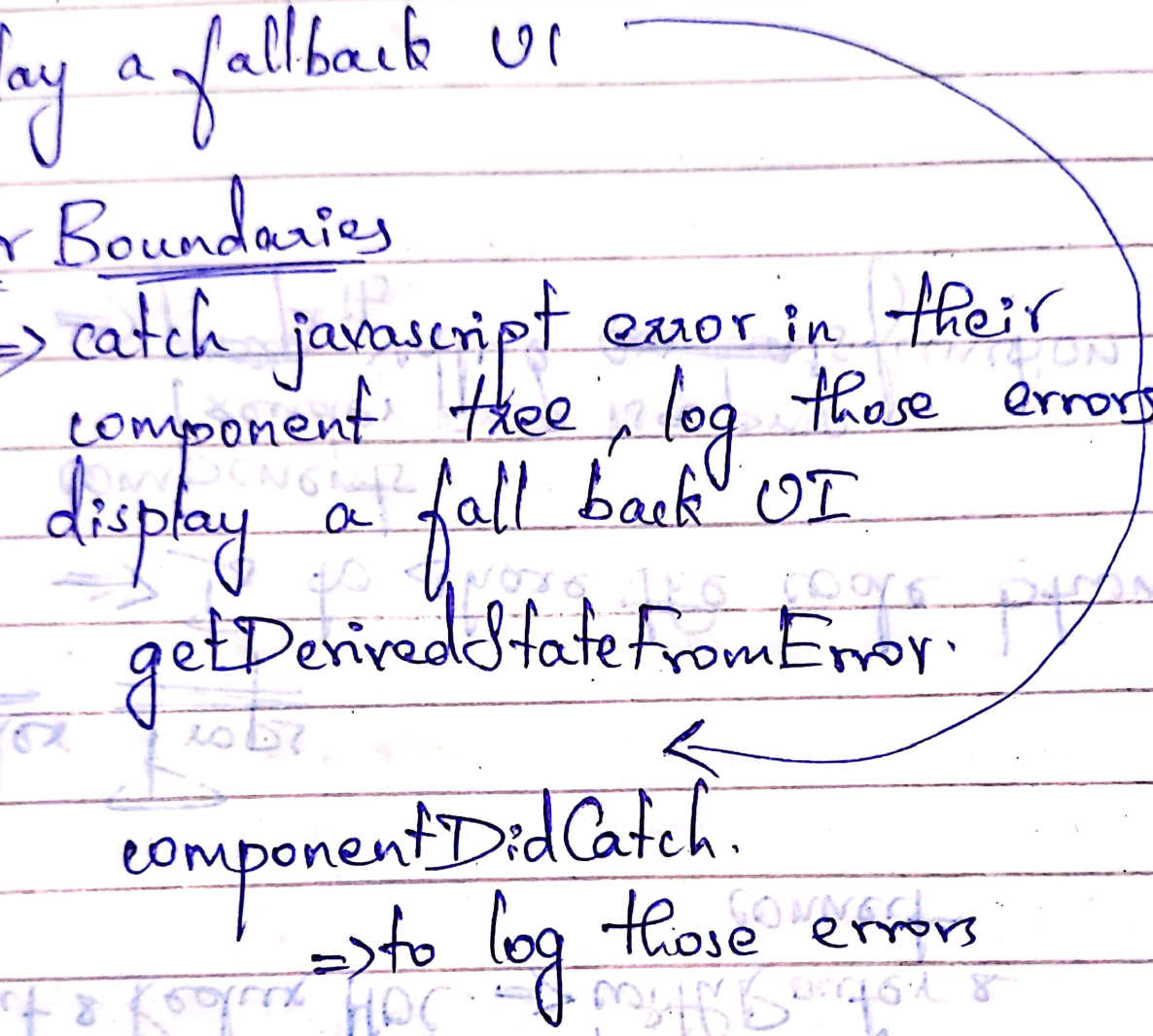
)

Error Boundaries

=> will catch the errors and display a fallback UI

Error Boundaries

=> catch javascript error in their child component tree, log those errors and display a fall back UI



Note:- Boundaries catch the error during rendering in lifecycle methods and in constructor of the whole tree below them.

Do not catch error inside event handler (need to use try catch).

Scanned with OKEN Scanner

Higher Order Components

Why → To share common functionality between components.

Definition → a fn takes an component as a argument & returns new component. ⇒ ① advance technique for reusing component logic.

React & Redux HOC. ⇒ withRouter & connect

Render Props

⇒ Is to share the code btwn the components.

⇒ ~~prop~~ render prop is a prop & its value ~~as~~ is function.

Optimizing Performance

Avoid Reconcilation.
Use the production build.
Do not mutate the data.

Strict mode

<React.StrictMode>

⇓

currently helps with

⇒ identifying components with unsafe lifecycles.

⇒ detecting legacy context API

⇒ warning about depreciated find DOMNode usage.

Lazy loading

=> is to identify resources as non-blocking and load these only when needed.

(or).

component or a part of code must be loaded whenever it is required.

<Suspense

=> not a data library.

=> a component is

=> mechanism for data fetching libraries to communicate to react that data a component is reading not ready yet.

Scanned with OKEN Scanner

Call, Apply

⇓

function borrowing.

call => first argument (reference of obj),
second argument (function parameter (arg1, arg2, ...))

apply => 1st argument (reference of obj),
2nd " (function as list of arguments).

Call & Apply => used to invoke the fn directly by passing

Bind

=> doesn't invoke the fn directly instead it'll create copy of the fn which can be invoked later.

Callback fn

A function passed into another function as a argument.

Promise

=> asyn call

=> resolve => then

=> reject => catch.

Asyn await.

=> ~~convert~~

=> await fn is used to wait for the promise.

=> It chaye asynchronous call to synchronous call (await).

=> Should be used within asyne block

Arrow fn

allows to create a fn in cleaner & simpler compared to regular fn.

Hoisting

=> Lifting variable declaration at the top scope.

=> we can acces these var & fn even before initialised.

=> In [var & fn] in global obj it ll allocate a memory. so it ll return undefined.

=> In [let & const] it ll also allocate a memory but in separate memory space. so can't acces this memory space before putting values in them

let & const

Blocked scope.

var

global object.

Local Storage

=> ~~doesn't~~ data doesn't expire

=> lasts until the user deletes it

SessionStorage

data in session storage ll be cleared whenever the page session ends

Callback function

=> is a function passed as an argument to another function.

=> a callback fn can run after another function has finished.

Hoisting

In g

We can access the variable before initialisation. In var & fn. with error.

But in let since it is a separate memory space we can't access before initialisation

It ll give reference.

Temporal Dead Zone.

=> Is the time since when let variable was hoisted. and till it is initialized to some value

=> that time btwn then

Reference Error

Variable inside temporal dead zone gives reference error.

Scanned with OKEN Scanner

Closure

⇒ is a fn bind together with its lexical environment.

(or)

fn along with lexical scope. is called closure.

(or)

combi of fn bundled together with reference to surrounding state (the lexical env)

useCallback

=> Not going to rerun the code inside of it unless certain certain parameters change

useMemo	useCallback
it takes a fn & it going to return you return only value of the fn.	it takes a fn, it will not just return the value of fn.

useMemo => ~~set~~ takes a fn and returns a value of fn

useCallback => takes a fn & returns a fn itself.

example for useMemo

```
function slowFunction(num) {
console.log('Called');
for(let i=0; i<=1000000; i++) {}
return num * 2;
}
```

```
const n = useMemo(
() => {
return
slowFunction(counter)
}, [counter])
```

Saga => - non - blocking task.
fork - useful when a saga needs to start a
Continue executing without waiting
for it to complete.

call	put
instructs middleware to call the promise	instructs middlewar to dispatch action to store.

Why middleware => third party extn btwn dispatching a action and the moments it reaches the reducer

split() => split a string obj into arr. of strings by separating the string into substring.

splice() => splice method changes the content of an array by removing existing elements or adding new elements /splice(start, deleteCount, item

slice() => returns a shallow copy of a portion of an array into a new array obj selected from start to end (not included)

Scanned with OKEN Scanner

Html => Hyper Text Markup Language.

React

React.lazy() => a component or a part of code must be loaded when it is required.

```
const Component = React.lazy(() =>
        import('./OtherComponent'));
```

```
<Suspense>
    <OtherComponent />
</Suspense>.
```

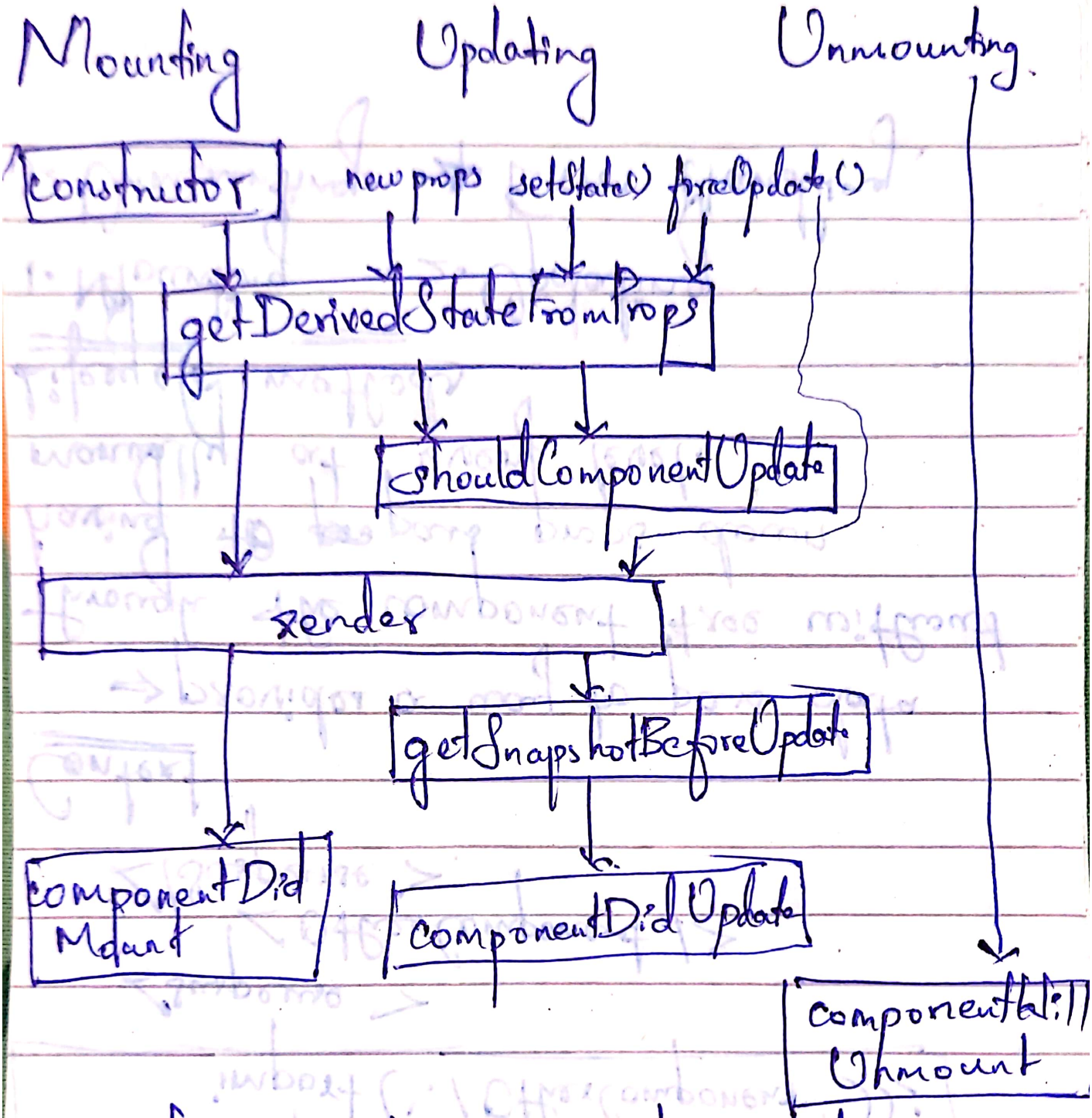
Context

=> provides a way to pass data through the component tree without having to ~~pass~~ pass props down manually at every level.

Lifecycle methods

1. Mounting
2. Updating
3. Unmounting
4. Error Handling.

Scanned with OKEN Scanner



mounting ⇒ when an instance of comp is being created & inserted into the DOM

Updating ⇒ component is being re-rendered as a result of changes to either its props or state.

Unmounting => when a component is being removed from the DOM.

componentDidMount => invoked immediately after a comp. & its children comp have been rendered to DOM.

componentDidUpdate => Called after the render is finished in the re-render cycle.

componentWillUnmount => method is invoked immediately before a comp is unmounted

Hooks

useRef => similar to useState
we can store previous value in it
persists btwn diff renders, but it is causing the comp. to re-render.

```
const value = useRef(0)
useEffect(() => {
  value.current = value.current + 1
})
```

usestate

allows us to track state in functional component.

one value for initiligation

another value is to update this value.

UselayoutEffect

to perform action before DOM rendering

it is synchronous it might cause performance issue.

(vs)

useEffect

it is asynchronous

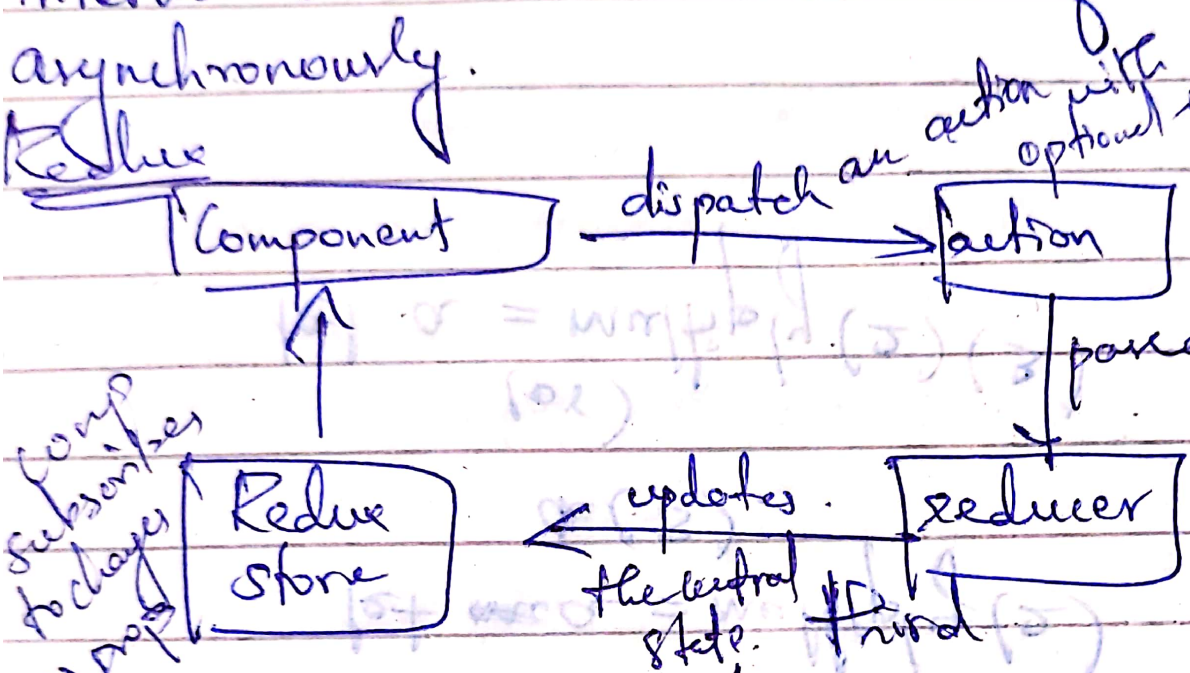
to perform action after DOM renders.

UseContext

used to create common data that can be accessed throughout the component hierarchy.

Scanned with OKEN Scanner

Redux Saga → is a middleware library used to allow a redux store to interact with resources outside of itself asynchronously.



Await

⇒ code should wait until this make request is finished & afterwards it'll execute the next thing

Async.

in async fn once js hits the await statement, it'll leave the fn do other work inside of program.

⇒ And then as soon as request finishes executing it'll come back into here return the result into response variable.

```
async function doWork() {
  cons response = await makeRequest('Google')
}
```

Scanned with OKEN Scanner

Currying

=> instead of taking all arguments at one item.

Takes first one & return a new fn

" second " " " " "

third " " " "

untill all arguments are completed.

Example :- let multiply = function (x) {
return function (y) {
console.log (x * y);
}
}

let a = multiply (2)
(3).

(or)

let a = multiply (2) (3).

Closure

=> A fn along with its lexical environment.

=> is a ~~closure~~ feature.

=> inner fn has access to outer fn. variable -> scope chain.

Closure has three scope chains.

=> has access to its own scope.

=> has access to outer function variable.

=> has access to global variable.

Router

=> enables the navigation among views of various components in react app.

=> helps create and navigate btwn diffent URLs that make up web application.

Scanned with OKEN Scanner

CSS

position : static => default value

position : relative => it follows the Html document flow.
=> but if we give top, right, left, ll work.

position : fixed => if we scrolls, it comes along with scroll
(always stays in the same place even scroll happens)

position : sticky => combination of relative and fixed

position : absolute => positioned relative to the nearest ancestor (instead of positioned viewport)
=> no positioned ancestor then it uses the document body and moves along with page scrolling.

setTimeout => executes a fn after waiting a specified number of milliseconds. (executes a fn after delay)

setInterval => calls a fn repeatedly with a delay between calls.

Spread

=> used to unpack iterables such as arrays, objects and function calls.

Rest

=> we can pass indefinite number of parameters to our fn.

Destructuring

=> allows us to extract data from arrays, objects and maps & set them into new distinct variables.

Scanned with OKEN Scanner

Redux Saga

on click of on UI (save button), acti

↓

action goes from UI & it should go to the updat server & save on to the server then only we can update the reducer.

from component if we disptach a action, it ll catch in the watcher class

1. Particular action ll be catched by watcher class, whenever dispatched is done from component. (start)
2. Once it catches it ll perform asyne action, once it finishes then it ll dispatch another action to reach the reducer ('succ

React Router ⇒ fully featured client & server side routing library for react

* helps to create & navigate btwn different URL
* provides diff url for diff components.

Protected Route

⇒ require a user being authorized to visit a route.

⇒ so if user is not authorized for a specific page they cannot access it.

Intercept

If I intercept the api req, & response

Axios Interceptors

⇒ allows us to manipulate the header, body, parameters of the requests sent to the server as well as check the response received from the server for the most reasonable

We can intercept requests or responses before they are handled by then or catch.

Scanned with OKEN Scanner